# JAILEDDOTFUN ENHANCEMENT
Security Review



JANUARY, 2026

www.chaindefenders.xyz
https://x.com/DefendersAudits

## Lead Auditors



PeterSR



0x539.eth

## Table of Contents


- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Commit Hash
  - Scope
  - Roles
- Executive Summary
  - Issues Found
  - Security Grade
- Findings
  - Medium
  - Informational


## Protocol Summary

A Solana-based pickaxe training and exploration game. It blends NFT-style assets, STEPN-like exploration loops, and a Token-2022 economy with JAIL. On-chain handles ownership and verification; off-chain handles computation and player experience.

This enhancement completely removes the pickaxe level system from the game, significantly simplifying the gameplay mechanics. It also introduces batch repair

functionality, allowing players to repair multiple pickaxes in a single transaction with significant efficiency improvements.

## Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

## Risk Classification

| Likelihood/Impact | High | Medium | Low |
|---|---|---|---|
| High | H | H/M | M |
| Medium | H/M | M | M/L |
| Low | M | M/L | L |

## Audit Details

### Commit Hash

0f5da6608d1d2bcc529d1df119603246633c3e05

## Scope

| Id | Files in scope |
|---|---|
| 1 | instructions/token/jail_token.rs |
| 2 | instructions/token/mod.rs |
| 3 | instructions/breeding/vrf.rs |
| 4 | instructions/breeding/pickaxe.rs |
| 5 | instructions/breeding/mod.rs |
| 6 | instructions/marketplace/mod.rs |
| 7 | instructions/marketplace/marketplace.rs |
| 8 | instructions/admin/mod.rs |
| 9 | instructions/admin/burn.rs |
| 10 | instructions/admin/admin.rs |
| 11 | instructions/exploration/start.rs |
| 12 | instructions/exploration/mod.rs |
| 13 | instructions/exploration/loadout.rs |
| 14 | instructions/exploration/rewards.rs |
| 15 | instructions/mod.rs |
| 16 | instructions/equipment/pickaxe_level.rs |
| 17 | instructions/equipment/gem_inventory.rs |
| 18 | instructions/equipment/initialize.rs |
| 19 | instructions/equipment/pickaxe_repair.rs |
| 20 | instructions/equipment/mod.rs |
| 21 | instructions/equipment/gem_level.rs |
| 22 | instructions/equipment/pickaxe_inventory.rs |
| 23 | instructions/equipment/gems.rs |
| 24 | instructions/boxes/mystery_rewards.rs |
| 25 | instructions/boxes/mod.rs |
| 26 | instructions/boxes/pickaxe_boxes.rs |
| 27 | utils/mod.rs |

| Id | Files in scope |
|---|---|
| 28 | utils/vrf_helpers.rs |
| 29 | utils/signature.rs |
| 30 | utils/shared.rs |
| 31 | constants.rs |
| 32 | heap.rs |
| 33 | lib.rs |
| 34 | state.rs |
| 35 | errors.rs |

Note: The scope of the audit was PR 26 and PR 27.

### Roles

| Id | Roles |
|---|---|
| 1 | Admin |
| 2 | Operator |
| 3 | User |

## Executive Summary

### Issues Found

| Severity | Count | Description |
|---|---|---|
| High | 0 | Critical vulnerabilities |
| Medium | 1 | Significant risks |
| Low | 0 | Minor issues with low impact |
| Informational | 2 | Best practices or suggestions |
| Gas | 0 | Optimization opportunities |

### Security Grade




Based on the audit findings and code quality, the overall protocol security grade is 98.50 / 100.00. This reflects an excellent security posture with small room for improvement.

## Findings

## Medium

## Mid 01 Cross-Cluster Or Cross-Deployment Replay Attack

### Location

pickaxe_repair.rs

### Description

The `pickaxe_repair.rs` contract implements an off-chain signed message verification mechanism for repairing pickaxes, using a signature from an admin

authority to authorize the transaction. However, the constructed message string for signature verification lacks proper domain separation. Specifically, it only includes elements like the player's public key, number of repairs, total JAIL token burn amount, nonce, expiration timestamp, and a hash of the repair items. It does not incorporate deployment-specific identifiers such as the Solana program ID or a cluster-specific value (e.g., genesis hash or a realm identifier like "devnet" or "mainnet").

This omission allows signatures generated in one Solana cluster (e.g., Devnet) to be replayed in another cluster (e.g., Mainnet) if the same `signature_authority` keypair is used across environments - a common practice for development and production deployments. As a result, an attacker could obtain a low-cost or free signature on a testnet and use it to perform unauthorized or subsidized actions on the mainnet, leading to economic exploits like free pickaxe repairs by burning minimal or valueless tokens.

The vulnerable code is in the message construction:

```
let message = format!(
    "repair_pickaxes_{}_{}_{}_{}_{}_{}",
    ctx.accounts.player.key(),
    repairs.len(),
    total_burn,
    nonce,
    expires_at,
    repairs_hash
);
```

This message is then verified against the admin signature without binding it to the specific program or cluster context.

## Proof of Concept

1. Assume the game developers deploy the same contract code to both Devnet and Mainnet, using the identical admin keypair for the `GameConfig`'s `signature_authority` in both environments (a standard operational setup to simplify management).
2. An attacker creates or uses a player profile on Devnet and requests a batch repair for, say, 10 pickaxes through the game's backend API. On Devnet, JAIL tokens are typically abundant, cheap, or even freely mintable for testing purposes, making the burn cost negligible.
3. The backend server validates the request and generates a signed message payload, including the signature, for the attacker's player public key, a

specific nonce (e.g., 100), the repair details (pickaxe IDs, HP values, burn amounts), total burn, and expiration time. This signature is returned to the attacker.

4. The attacker then constructs a transaction with the exact same payload - including the repairs vector, nonce, expiration, and signature - and submits it to the Mainnet cluster.
5. On Mainnet, the contract reconstructs the message string identically because it lacks any program ID or cluster-specific elements. If the attacker's Mainnet player profile has not yet used nonce 100, the nonce check passes, the signature verifies against the shared admin authority, and the expiration is still valid.
6. The transaction executes successfully on Mainnet, repairing the pickaxes and burning JAIL tokens from the attacker's Mainnet account. However, since the signature was obtained cheaply on Devnet, the attacker effectively performs the repair at a fraction of the intended cost or for free, exploiting the economic model of the game. This can be repeated for multiple repairs or scaled by coordinating nonces, potentially draining value from the Mainnet economy.

This attack is feasible because Solana's signature verification (using ed25519) is deterministic and cluster-agnostic, relying solely on the message bytes and public key.

## Recommendation

To prevent cross-cluster replay attacks, implement domain separation by incorporating deployment-specific identifiers into the signed message string. This binds the signature to a particular program instance and cluster, making it invalid when replayed elsewhere.

Specifically:

- Include the program ID to tie the message to the exact deployed program.
- Add a cluster-specific realm identifier, such as a string like "mainnet", "devnet", or "staging", which could be stored in the `GameConfig` account or hardcoded per deployment. Alternatively, use the cluster's genesis hash if dynamically accessible, but a configurable realm is simpler for multi-environment setups.

### Status

Fixed

Response: Added program ID to signature messages to prevent cross-cluster/cross-deployment replay attacks.

## Informational

### Info 01 `burn_jail_if_needed` Executed Per Pickaxe Instead Of Once For Total Amount In Batch Repair

#### Location

pickaxe_repair.rs

#### Description

The current implementation of the `repair_pickaxes` instruction burns JAIL tokens separately for each pickaxe inside the repair loop by calling `burn_jail_if_needed( ... )` multiple times (once per item in `repairs` vec).

However, the function already accumulates the total burn amount (`total_burn`) earlier in the instruction and the admin-signed message also includes this total amount (`total_burn`), which means the intention is to perform one aggregated burn transaction for the whole batch repair.

#### Recommendation

Remove the redundant `burn_jail_if_needed` call inside the `for repair in repairs` loop and add only single burn that uses the total amount.

#### Status

Fixed

# Info 02 Wrong Game Design Documentation Regarding Pickaxe Levels

## Location

game-design-en.md

## Description

The game design documentation references pickaxe levels (0–10) and level-dependent mechanics (gem slot unlocks at levels 3, 5, 8, 10; breeding requirement of Lv5+), but pickaxe levels have been removed from the actual implementation. This creates a mismatch between the documented design and the current codebase, potentially misleading developers, auditors, and players about how the game mechanics actually function.

## Recommendation

Update the documentation to remove all references to pickaxe levels as they don't exist anymore.

## Status

Fixed